

Global Solution 2026

FIAP - Faculdade de Informática e Administração Paulista
Curso de Ciência da Computação (On-Line)
Relatório Técnico

1. Introdução e Análise do Cenário

Quando paramos para pensar em missões espaciais, o tamanho do desafio tecnológico é enorme. Se monitorar e interpretar os dados a partir do planeta Terra já é complexo, fazer isso a partir de um outro planeta, no espaço, eleva o nível de complexidade ao extremo.

As condições operacionais mudam rapidamente e a comunicação com a base na Terra sofre com atrasos críticos (latência) ou simplesmente é perdida, gerando risco para as missões espaciais. Pensando nisso, nossa equipe percebeu que o sistema de telemetria não poderia ser somente um sistema passivo, que lê os dados e apresenta-os em um painel; ele precisa ser inteligente o suficiente para ler os dados, interpretá-los e gerar informações que auxiliem os astronautas a tomarem as melhores decisões durante a missão, e tudo de uma maneira local, com nenhuma ou a mínima dependência de informações externas.

Para que o nosso sistema simule as condições de uma missão espacial de maneira mais realista possível, foi desenhado um cenário baseado em seis módulos vitais para a base aeroespacial. No conjunto de dados simulados que serve como ponto de partida para a nossa análise, definimos o seguinte panorama operacional:

Os Módulos de Suporte de Vida, Energia, Habitat e Armazenamento estão operando normalmente (status 1), enquanto os módulos de Comunicação e de Laboratório apresentam falhas ou estão fora do ar (status 0).

Com esse cenário inicial de instabilidade onde a comunicação está cortada e o laboratório está fora do ar, nossa intenção foi mostrar como o software recebe e processa esse fluxo constante de dados dos sensores em tempo real. A partir daí, o sistema precisa aplicar regras lógicas para entender o tamanho do problema, organizar os alertas na fila por prioridade, prever o comportamento da bateria e disparar as ações necessárias de forma totalmente automática. No fim das contas, o grande objetivo do nosso grupo foi desenvolver uma lógica que consiga equilibrar a geração e o consumo de energia para manter a base funcionando e os astronautas seguros, mesmo quando tudo no ambiente externo parece estar jogando contra a missão.

2. Engenharia e Estruturas de Dados

Para oferecer alta eficiência computacional ao sistema, previsibilidade de tempo de execução e aderência aos requisitos fundamentais do pensamento computacional, a escolha das estruturas de dados foi motivada pela necessidade de representar as informações da colônia.

2.1. Modelagem das Estruturas e Justificativas

- **Dicionários (Tabelas Hash):**

Utilizados no mapeamento de estados e configurações do sistema, como visto na chave "modulos" do arquivo "data.json". A escolha desta estrutura foi motivada pela alta eficiência que ela oferece ao sistema, dado que ela garante complexidade média de busca e atualização em tempo constante, $O(1)$. Isso permite que o módulo de diagnóstico no arquivo "diagnostico.py" verifique se um componente como o "suporte_vida" ou "comunicacao" está ativo ou não de forma instantânea, eliminando completamente a necessidade de varrer a memória RAM de forma linear, $O(n)$.

- **Listas (Vetores Dinâmicos):**

Utilizadas massivamente para o armazenamento e manipulação de dados cronológicos da colônia, como visto nas chaves do bloco "energia" do arquivo "data.json". A escolha desta estrutura foi motivada pela alta eficiência que ela oferece ao sistema, dado que ela garante indexação direta e acesso a qualquer registro por meio de seu índice em tempo constante, $O(1)$. Isso permite que o módulo de regressão no arquivo "regressao.py" alimente os laços de repetição que calculam as previsões de geração de energia de forma instantânea, eliminando a necessidade de buscas sequenciais para operações matemáticas.

- **Filas (Queue - FIFO):**

Utilizadas para o processamento ordenado de alertas pendentes da colônia, como visto na manipulação da lista "eventos" do arquivo "data.json". A escolha desta estrutura foi motivada pela alta eficiência que ela oferece ao sistema, dado que ela garante o tratamento cronológico dos eventos de forma justa e ordenada (Primeiro a Entrar, Primeiro a Sair). Isso permite que o módulo de logs no arquivo "logs_eventos.py" possa extrair e tratar o alerta mais antigo de forma sequencial, garantindo que nenhuma anomalia inicial seja negligenciada.

- **Pilhas (Stack - LIFO):**

Utilizadas para acesso imediato aos registros mais recentes da colônia, como visto na manipulação da lista "eventos" do arquivo "data.json". A escolha desta estrutura foi motivada pela alta eficiência que ela oferece ao sistema, dado que ela garante que o último evento registrado seja acessado em tempo constante, $O(1)$. Isso permite que o módulo de logs no arquivo "logs_eventos.py" consiga remover e analisar diretamente o topo da pilha, oferecendo agilidade máxima no diagnóstico da falha mais recente.

3. Regras Lógicas de Diagnóstico e Classificação de Status

A tomada de decisão do sistema foi construída com base em estruturas condicionais utilizando

a lógica booleana. O programa processa as informações lidas do arquivo "data.json" e avalia a situação da colônia comparando os dados dos sensores e o estado de funcionamento dos módulos.

3.1. Condições de Classificação do Consumo Técnico

No arquivo "balanceamento_energetico.py", o status de segurança energética da colônia é determinado através de verificações lógicas combinadas:

CRÍTICO MAX = Ocorre quando o consumo é maior que a geração solar ($\text{consumo}[i] > \text{geracao}[i]$) **E** a bateria está abaixo de 40% ($\text{bateria}[i] < 40$).

CRÍTICO = Ocorre se o consumo for maior que a geração solar ($\text{consumo}[i] > \text{geracao}[i]$) **OU** se o nível da bateria for menor ou igual a 60% ($\text{bateria}[i] \leq 60$).

ESTÁVEL = Atendido quando as condições anteriores não são disparadas, significando que o sistema opera dentro dos limites nominais seguros.

3.2. Mecanismo de Inferência (IF / ELIF / ELSE)

A escolha de implementar essa lógica por meio da estrutura if / elif / else no arquivo "balanceamento_energetico.py" foi motivada pela necessidade de garantir que o computador avalie as regras de forma excludente e prioritária. O sistema testa primeiro a condição mais severa "CRÍTICO MAX", passa para a intermediária "CRÍTICO" e apenas define como "ESTÁVEL" se nenhuma anomalia for detectada, garantindo total previsibilidade no fluxo de execução do software.

4. Algoritmo de Previsão Matemática e Tendência

Para estimar a produção de energia futura sem depender de bibliotecas externas pesadas, a escolha do algoritmo foi motivada pela aplicação prática de conceitos matemáticos estatísticos. No arquivo "regressao.py", foi implementado um modelo de Regressão Linear Simples através da função "prever_energia_eolica()".

O algoritmo utiliza laços de repetição "FOR" para percorrer os dados das listas de históricos de vento e de geração solar, calculando as somatórias necessárias para obter os coeficientes "a" e "b" da reta matemática. A partir da velocidade atual captada pelo sensor (vento_atual), o sistema realiza uma projeção através da fórmula:

$$\text{previsao} = (a * \text{vento_atual}) + b$$

Essa abordagem oferece uma previsão clara e rápida da capacidade de geração de energia da colônia diretamente na interface gerada no arquivo "sistema.py".

5. Simulação do Log de Eventos e Inconsistência

Proposital

Para a geração do histórico da colônia, o sistema realiza a leitura de 8 registros de atividades guardados na chave "eventos" do arquivo "data.json". No arquivo "logs_eventos.py", esses dados são carregados para montar um relatório detalhado no terminal do operador, onde são organizados para demonstrar na prática o uso de estruturas de dados.

A escolha de manipular a lista de eventos através dos métodos `.pop(0)` e `.pop()` foi motivada pela necessidade de mostrar como acessar os extremos de uma lista de forma direta. O método `.pop(0)` remove o primeiro item da lista, fazendo o papel de uma Fila para simular o tratamento do evento mais antigo registrado, enquanto o método `.pop()` retira o último item, fazendo o papel de uma Pilha para simular a análise da ocorrência mais recente. Junto com isso, o arquivo monta tabelas organizadas para exibir quais módulos estão desligados, como: "comunicacao" e "laboratorio" e as informações atuais do bloco "ambiente", mostrando na tela os últimos valores de temperatura, radiação e velocidade do vento.

6. Recomendações Técnicas Baseadas em Prioridades

A tabela abaixo consolida as ações e status que o operador do sistema visualiza ao interagir com a interface no arquivo "sistema.py", cruzando as saídas geradas por cada módulo de diagnóstico:

Módulo de Origem	Condição / Gatilho do Sensor	Exibição na Interface do Operador
balanceamento_energetic o.py	Consumo excedente com bateria abaixo de 40%	Classificação de status "CRÍTICO MAX" na tabela de consumo
balanceamento_energetic o.py	Consumo maior que geração ou bateria $\leq 60\%$	Classificação de status "CRÍTICO" na tabela de consumo
diagnostico.py	Valor do módulo mapeado como diferente de 1	Registro de situação como "DESLIGADO" no painel do operador
logs_eventos.py	Execução da análise estrutural de dados	Demonstração do tratamento sequencial via Fila e Pilha no terminal

7. Conclusões e Aprendizados

Desenvolver o Sistema Operacional Aurora ajudou a entender como aplicar na prática o que foi passado nas aulas de lógica de programação e estruturas de dados durante o semestre. Usar dicionários para checar de maneira rápida se os módulos estavam ligados ou desligados, e

usar listas para guardar o histórico dos sensores, mostrou como escolher a estrutura de dados mais adequada deixa o programa muito mais organizado e eficiente.

Criar as regras de decisão com estruturas condicionais como: **if**, **else** e **elif**, além de desenvolver o cálculo matemático da previsão diretamente no código, sem utilizar bibliotecas prontas, reforçou como a programação ajuda a dividir problemas grandes em partes menores para encontrar a solução.

8. Link do GitHub

<https://github.com/PedroGesini/Aurora-Operational-Control-System>